IT404041 REAL TIME PROGRAMMING · ASSIGNMENT 2 · SMART GREENHOUSE CONTROLLER

Role Guide

What each role did, how it works, and how to present it

Generated 2026-11-05

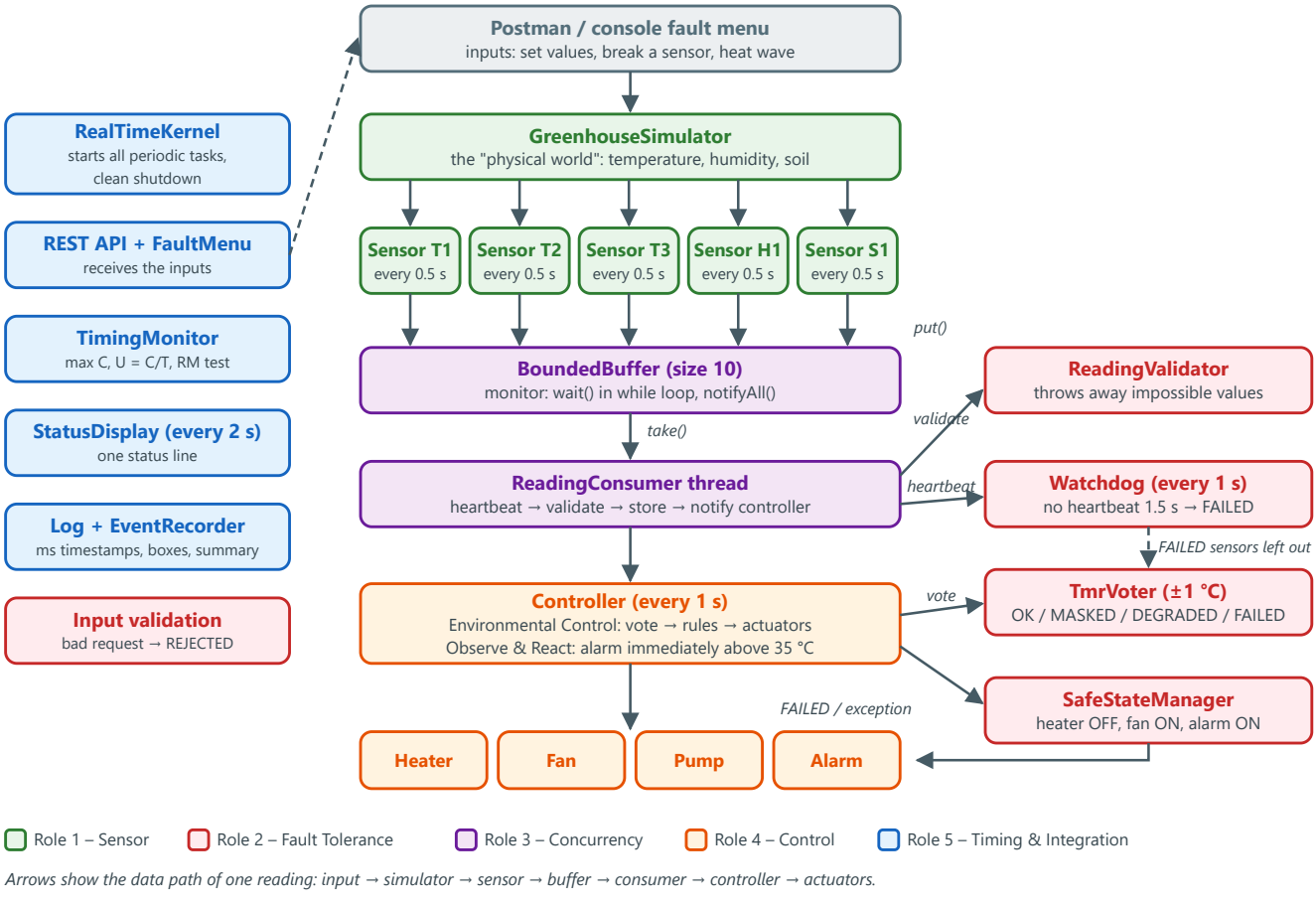
This guide is for the 5 team members. For each role it explains:

1. **The job** – what the role is responsible for.
2. **The code** – which files belong to the role.
3. **How it works** – the process, step by step.
4. **What we built** – a short "what I did" you can say in the presentation.
5. **The demo** – which Postman folder or menu key to run, and what the CLI will show.
6. **Questions the lecturer may ask** – with short answers.

The exact file list, hand-over points and report tables are in CONTRIBUTIONS.md.

0. The whole system in one picture (everyone should know this)

System architecture – which role owns which part



Arrows show the data path of one reading: input → simulator → sensor → buffer → consumer → controller → actuators.

The 5 roles at a glance

ROLE 1 Sensor Engineer	ROLE 2 Fault Tolerance	ROLE 3 Concurrency	ROLE 4 Control	ROLE 5 Timing & Integration
Produces the input	*Faults never harm plants*	*Safe data sharing*	*Decides the actions*	*Runs & measures all*

Code
`sim/`
`Sensor, FaultMode, GreenhouseSimulator`
	Code
	`fault/ + validation`
	`TmrVoter, Watchdog, SafeStateManager`
		Code
		`buffer/, demo/`
		`BoundedBuffer, ReadingConsumer`
			Code
			`control/`
			`Controller, Actuator`
				Code
				`kernel/, monitor/, api/`
				`RealTimeKernel, TimingMonitor, Log`

Postman folders F1, X2	**Postman folders** F2, F3, X1	**Postman folders** F4	**Postman folders** 01-Normal, F5, X3	**Postman folders** 00, 02, 03, X4
Lecture concepts 1, 2, 6	**Lecture concepts** 2, 3, 4, 5	**Lecture concepts** 7, 8, 9	**Lecture concepts** 1, 10	**Lecture concepts** 6, 11

One sentence for the whole project:

> "Five simulated sensors are read every 500 ms, the readings go through a monitor buffer to the controller, the three temperature sensors are combined by a TMR voter, a watchdog detects dead sensors, and whenever the controller cannot trust its data – or an exception happens – it switches the greenhouse to a safe state, all while every task's execution time is measured for a Rate-Monotonic test."

Role 1 – Sensor Engineer

Job

Produce the input of the whole system: simulate the greenhouse, and build the sensors that read it periodically. The sensors can be made faulty on command.

Code

File	Purpose
`sim/Sensor.java`	One sensor = one periodic task (`Runnable`), every 500 ms
`sim/FaultMode.java`	`NONE` , `WRONG_VALUE` , `STUCK` , `DEAD`
`sim/SensorType.java`	Temperature, humidity, soil
`sim/Reading.java`	One reading (sensor id, type, value, time). Immutable.
`sim/GreenhouseSimulator.java`	The physical world and its simple physics
Postman	`F1-WrongValue` , `X2-StuckAndImpossibleValue`

How it works (process)

1. **Simulation.** `GreenhouseSimulator` holds the true temperature, humidity and soil moisture. Every second it changes them:
   - the sun +0.01 °C, the heater +0.5 °C, the fan −0.5 °C and −1 % humidity;
   - the pump +2 % soil, and the soil dries −0.1 %.

   Postman can set the values directly or add a heat wave.
2. **Periodic sensor.** The kernel releases each `Sensor` every 500 ms (RT1, firm deadline). In `run()` the sensor:
   1. reads the simulator (true value + small noise ±0.2 °C);
   2. applies its fault mode;
   3. puts a `Reading` into the bounded buffer.
3. **Fault modes** (fault → error):
   - `WRONG_VALUE` : real value + offset (F1: +15 °C → T2 reports about 40 °C).
   - `STUCK` : always reports the value it had when the fault was injected.
   - `DEAD` : sends nothing at all, so the watchdog must notice.
4. **Thread safety.** The fault is set by the REST/menu thread and read by the sensor thread, so it is stored in a `volatile` field that holds one immutable object. Mode and value always change together.
5. **Never block, never die.**
   - A sensor waits at most 50 ms for space in the buffer, so it cannot miss its period.
   - Every exception is caught, because one uncaught exception would stop the periodic task forever.

```
// Sensor.sampleOnce()
FaultConfig f = fault;                    // read the volatile once
if (f.mode() == FaultMode.DEAD) return;    // dead: produce nothing
double value = switch (f.mode()) {
    case WRONG_VALUE -> simulator.read(type) + f.value();
    case STUCK       -> stuckAt;
    default          -> simulator.read(type);
};
buffer.put(new Reading(id, type, value, System.currentTimeMillis()), PUT_TIMEOUT_MS);
```

What I did (say this)

> "I built the greenhouse simulation and the five sensors. Each sensor is a Runnable released every 500 ms by the ScheduledExecutorService. I implemented three fault modes – wrong value, stuck and dead – that can be injected while the system runs. A faulty sensor only produces an *error*, a wrong or missing reading. The other roles make sure this error never becomes a *failure*, a wrong action of the greenhouse."

Demo

- **Postman:** run folder **F1-WrongValue**, or press menu key `1`.
- **CLI shows:**

```
>>> FAULT injected: T2 -> WRONG_VALUE (+15.0 C)
>>> OK -> MASKED : T2=40.0 C out-voted (fault masked), T1 and T3 agree -> voted T=24.9 C
```

  The status line shows `[MASKED 2/3]`, and the alarm and fan stay `off`. There is no false alarm.
- **Point at the code:** `Sensor.sampleOnce()`.

What the CLI looks like:

Smart Greenhouse Controller – CLI (F1: T2 wrong value)

```
================================== NEW INPUT RECEIVED ==================================
  11:50:04.221  from Postman:  POST /api/sensors/T2/fault  {type=WRONG_VALUE, value=15.0}
  WHAT IT MEANS : temperature sensor T2 will report a WRONG value (+15.0 added to the real value)
  EXPECTED      : voter out-votes T2 -> MASKED, no wrong action
  SYSTEM REACTION (lines below):
----------------------------------------------------------------------------------------
11:50:04.222 WARN  [FAULT     ] >>> FAULT injected: T2 -> WRONG_VALUE (+15.0 C)
11:50:04.224 WARN  [VOTER     ] >>> OK -> MASKED : T2=39.9 C out-voted, T1 and T3 agree -> T=25.0 C
11:50:05.225 INFO  [STATUS    ] (every 2 s) T=25.0C [MASKED 2/3] Heater off Fan off Alarm off | T2:WRONG_VALUE
11:50:06.313 INFO  [CHECK     ] Postman reads the status (test step) -> vote MASKED, safe state no

================================== NEW INPUT RECEIVED ==================================
  11:50:06.408  from Postman:  DELETE /api/sensors/T2/fault
  WHAT IT MEANS : temperature sensor T2 is repaired
----------------------------------------------------------------------------------------
11:50:06.408 INFO  [FAULT     ] +++ T2 repaired - sensor works normally again
11:50:07.225 INFO  [VOTER     ] +++ MASKED -> OK : all 3 agree (T1=25.2, T2=24.9, T3=25.1)
```

Questions the lecturer may ask

Question	Answer
Why `scheduleAtFixedRate` and not `Thread.sleep(500)`?	Fixed rate releases the task every 500 ms from the start time. Sleep would add the execution time to every period and drift.
What is the difference between fault, error and failure?	Fault = the cause (broken sensor). Error = wrong internal state (wrong reading). Failure = wrong visible behaviour (heater ON at 40 °C).
Why is `Reading` a record?	It is immutable, so it can be passed from the sensor thread to the consumer thread safely without locks.

Report section

Sensors, periodic tasks (RT1), and the fault → error → failure table (in CONTRIBUTIONS.md, Role 1).

Role 2 – Fault Tolerance Engineer

Job

Make sure a faulty sensor or a software error never harms the plants. Detect, mask and degrade, and if nothing can be trusted, go to a safe state.

Code

File	Purpose
`fault/TmrVoter.java`	TMR majority voter, ±1 °C
`fault/VoteStatus.java`, `fault/VoteResult.java`	OK / MASKED / DEGRADED / FAILED
`fault/Watchdog.java`	Heartbeat check every 1 s, timeout 1.5 s
`fault/SafeStateManager.java`	Safe state, automatic exit, escalation
`fault/ReadingValidator.java`	Rejects impossible readings
`api/GlobalExceptionHandler.java`, `api/dto/*Request.java` (3), `kernel/UnknownSensorException.java`	Rejects invalid inputs from Postman
Postman	`F2-DeadSensor`, `F3-SafeState`, `X1-InvalidInputs`

How it works (process)

The faults go through 4 layers of defence:

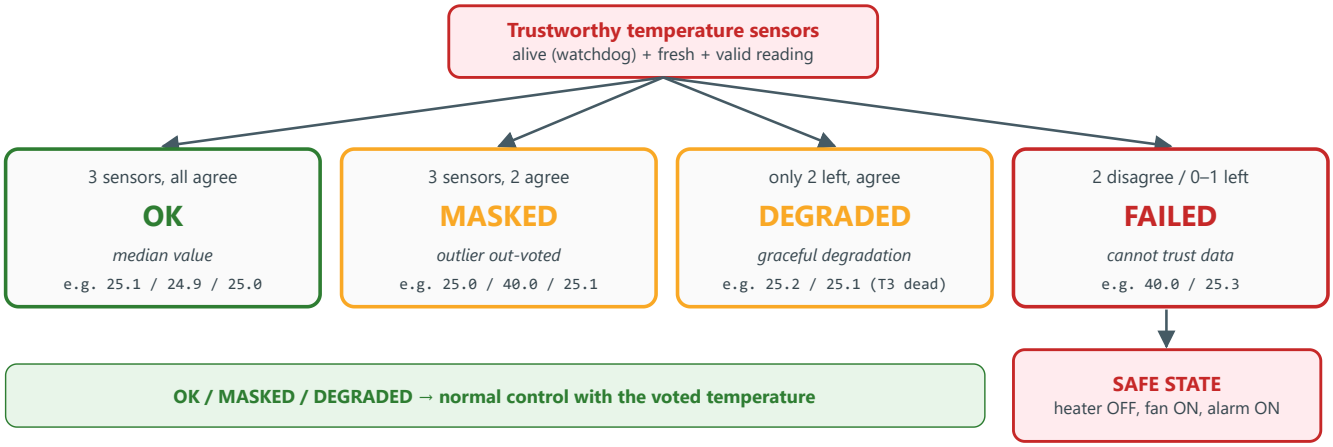
1. **Input validation.** Wrong REST input is rejected with 400/404 before it reaches the system. Examples: temperature 500, text instead of a number, unknown sensor `T9`.
2. **Reading validation.** A reading of NaN or outside −40…80 °C / 0…100 % is thrown away, and the sensor is not trusted.
3. **Watchdog (heartbeat).**
   - Every reading that arrives is a heartbeat.
   - Every 1 s the watchdog checks the age of the last heartbeat. Older than 1.5 s → **FAILED**. Heartbeats back → **RECOVERED**.
   - Worst case: 1.5 s timeout + 1 s check period = **2.5 s**, which meets the hard deadline RT3.
4. **TMR voter (inexact voting).** Two values agree if they differ by at most 1 °C.

Sensors	Result
3 agree	**OK**
2 agree, 1 outlier	**MASKED** (the fault is hidden)
only 2 left and they agree	**DEGRADED** (graceful degradation)
anything else	**FAILED**

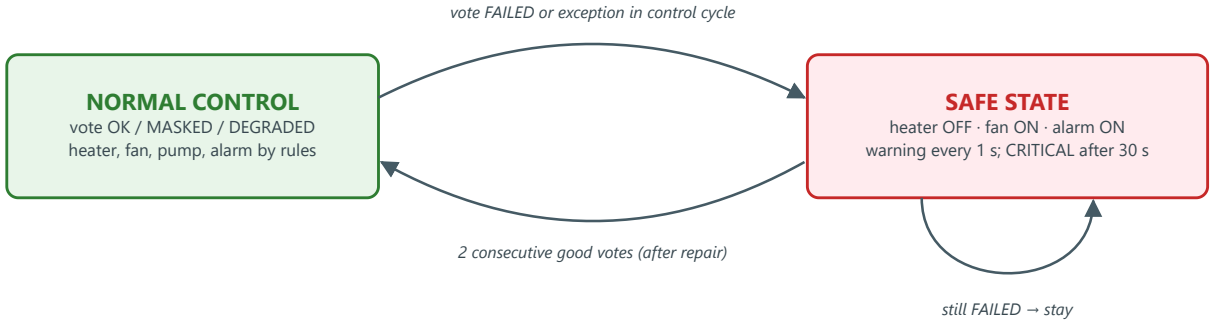
5. **Safe state (fail-safe).**
   - **When:** the vote is FAILED, or an exception happens in the control cycle.
   - **Action:** heater OFF, fan ON, alarm ON. The plants cannot overheat.

- **Repeated warning:** `SAFE STATE ACTIVE` is logged every cycle. After 30 s, `CRITICAL – manual intervention` is logged every 2 s.
- **Leaving it:** automatic after **2 consecutive good votes**, so one lucky reading is not enough.
- **No self-stop:** the system never stops itself, because stopping would also switch the fan off.

TMR voter – inexact voting (values agree within ±1 °C)



Safe state – fail-safe behaviour and forward recovery



Why these outputs? Without a trusted temperature the worst danger is overheating: cooling and warning a human is the safe side.
The system never stops itself – stopping would also switch the fan off.

```
// Watchdog.check()
long age = now - latest.lastHeartbeat(id);       // time since last heartbeat
if (age > TIMEOUT_MS && !wasFailed) {             // 1500 ms
    setFailed(id, true);                          // → voter gets only 2 sensors
    ...  "[RT3 HARD] T3 detected dead 1611 ms after it stopped -> MET"
} else if (age <= TIMEOUT_MS && wasFailed) {
    setFailed(id, false);                         // RECOVERED
}
```

What I did (say this)

> "I built the fault-tolerance layers. The TMR voter compares the three temperature sensors with ±1 °C tolerance, so one wrong sensor is masked and one dead sensor only degrades the system. The watchdog detects a dead sensor from its missing heartbeats within 2.5 seconds. When no majority is possible, or the control cycle throws an exception, the safe state switches heater off, fan and alarm on, and leaves automatically after two good votes. I also made sure invalid inputs from outside are rejected instead of crashing the system."

Demo

- **F2:** Postman **F2-DeadSensor**, or menu `2`. CLI:

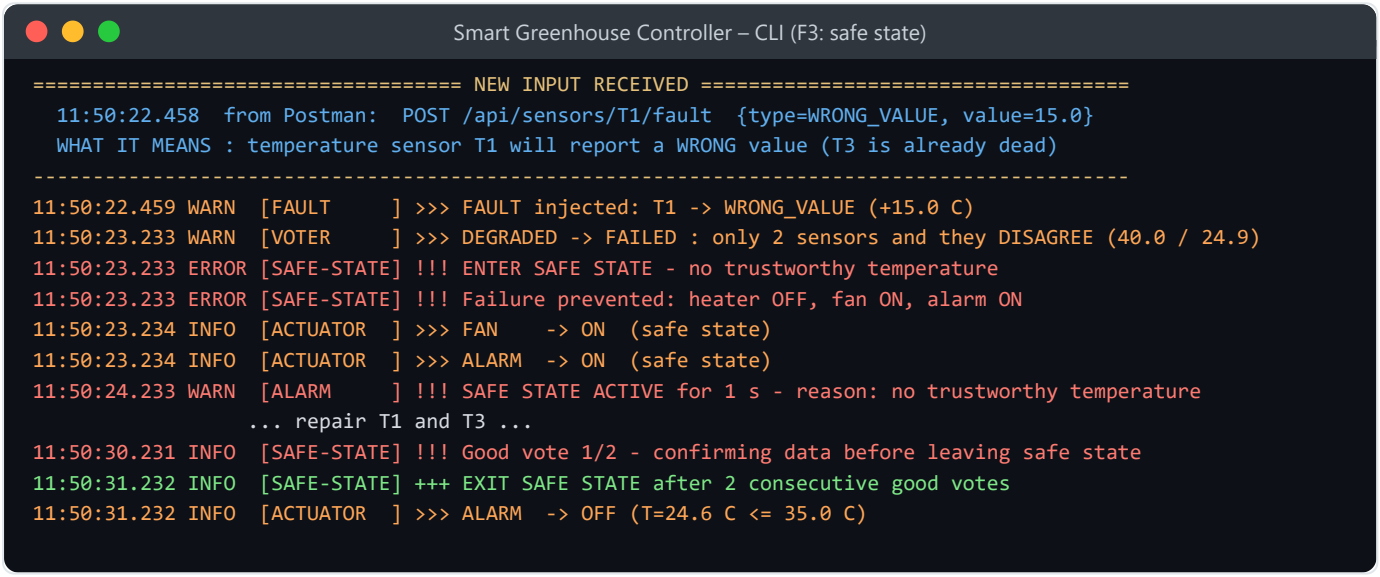
```
!!! T3 FAILED - no heartbeat for 2001 ms (timeout 1500 ms)
>>> [RT3 HARD] T3 detected dead 1611 ms after it stopped -> MET (deadline 2500 ms)
>>> OK -> DEGRADED : only 2 sensors (T1=25.2, T2=25.1) - they agree
```

- **F3:** Postman **F3-SafeState**, or menu `3` and later `6` to repair. CLI:

```
>>> DEGRADED -> FAILED : only 2 sensors and they DISAGREE (T1=40.1, T2=25.3)
!!! ENTER SAFE STATE - no trustworthy temperature ...
!!! SAFE STATE ACTIVE for 0 s ... (repeated x13)
+++ EXIT SAFE STATE after 2 consecutive good votes
```

- **X1-InvalidInputs:** 12 wrong requests, each answered with `REJECTED ... - system state unchanged`.
- **Point at the code:** `TmrVoter.vote()`, `Watchdog.check()`, `SafeStateManager.enter()`.

What the CLI looks like:



Questions the lecturer may ask

Question	Answer
Why ±1 °C and not exact equality?	Real sensors never give identical numbers (noise). This is inexact voting.
Why is "heater OFF, fan ON, alarm ON" safe?	The worst danger is overheating. With no trusted temperature, cooling and warning a human is the safe choice.
Why 2 good votes before leaving the safe state?	To avoid jumping in and out on one lucky reading (flapping).
What is forward recovery?	We don't go back to an old state. We catch the exception, move to a known safe state, and continue with the next cycle.
Why can the watchdog take up to 2.5 s?	Timeout 1.5 s + up to one check period of 1 s.

Question	Answer
Was RT3 ever missed?	Yes, once by 3 ms (2503 ms). The worst case 1.5 s + 1.0 s is **exactly** the 2.5 s deadline, so there is **zero margin**. A few ms of garbage collection or Windows scheduling delay pushes it over. We kept the assignment's 1000 ms watchdog period and report this honestly. The CLI prints `MISSED by 3 ms`, and the Postman test allows 100 ms jitter tolerance. **Fix in a real system:** check every 0.5 s (worst case 2.0 s), or use a real-time OS.

Report section

Redundancy, TMR voting, graceful degradation, watchdog, safe state (voting table in CONTRIBUTIONS.md, Role 2).

Role 3 – Concurrency Engineer

Job

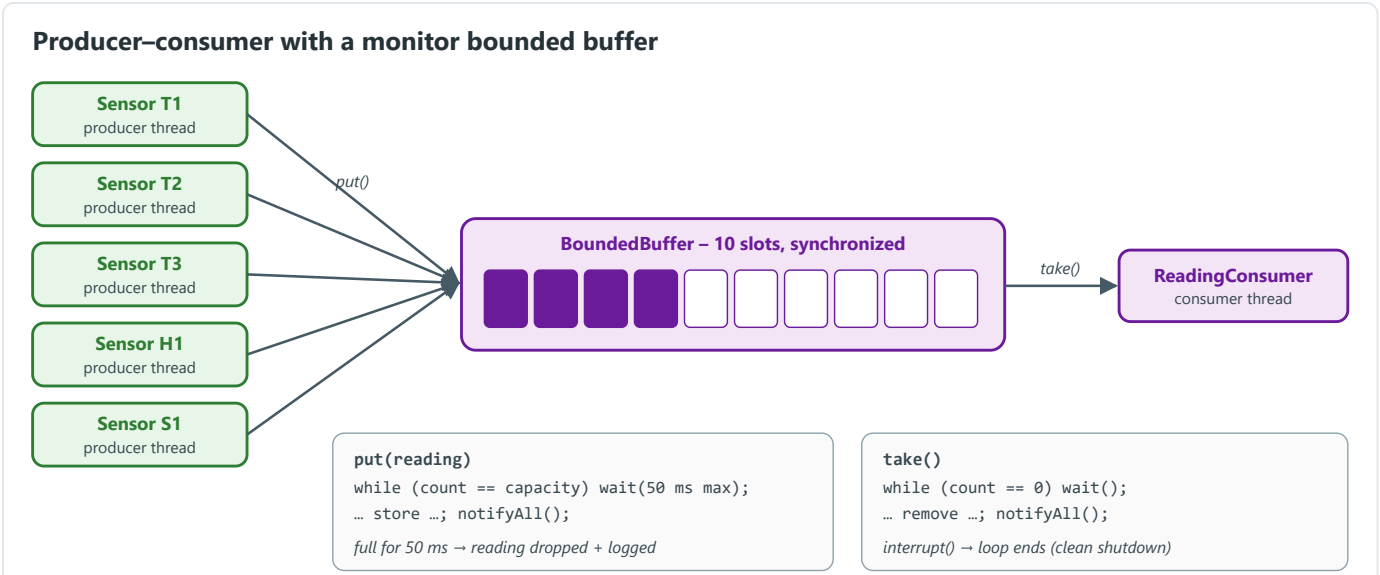
Many threads share data in this system. Role 3 makes sure the sharing is correct: a hand-written monitor buffer, and correct use of `synchronized`, `volatile` and atomic variables everywhere. Role 3 also shows a race condition and its two fixes.

Code

File	Purpose
`buffer/BoundedBuffer.java`	Own monitor buffer, size 10 (no BlockingQueue)
`buffer/ReadingConsumer.java`	Consumer thread
`buffer/LatestReadings.java`	Shared latest readings and heartbeats
`demo/RaceConditionDemo.java`	F4 race demo
Postman	`F4-RaceCondition`
Also reviewed	The shared fields in all other roles' files (shared-data table)

How it works (process)

1. **Producer–consumer.** 5 sensor threads (producers) `put()` readings, and 1 consumer thread `take()`s them.
2. **Monitor rules:**
   - Every method is `synchronized`, so only one thread is inside at a time.
   - Wait with `while (condition) wait();`, **never** `if`. After waking up the condition must be checked again (spurious wake-ups, other threads were faster).
   - After every change, `notifyAll()` wakes the threads waiting on the other side.
   - `put()` waits at most 50 ms. If the buffer is still full the reading is dropped and logged, because a sensor must not miss its period.
3. **Consumer thread.**
   - For each reading: heartbeat → validate → store → Observe & React.
   - **Clean stop:** the kernel calls `interrupt()`. `wait()` throws `InterruptedException`, the loop ends, and the CLI shows `Consumer thread stopped`.
4. **Shared data protection.**
   - `volatile` for simple flags read by other threads (fault setting, actuator state, safe-state flag).
   - `synchronized` for data that changes together.
   - `AtomicLong` / `AtomicBoolean` for counters and one-shot flags.
5. **F4 race demo.** Two threads each do `count++` 100 000 times. `count++` is read → add → write (3 steps), so the threads overwrite each other.
   - **Fix 1:** `synchronized increment()`.
   - **Fix 2:** `AtomicInteger.incrementAndGet()` (lock-free compare-and-set).



"while" and not "if": after waking up, the condition must be checked again (spurious wake-ups, other threads were faster).

```
// BoundedBuffer.take()
public synchronized Reading take() throws InterruptedException {
    while (count == 0) {
        wait();                    // buffer empty → sleep, releases the lock
    }
    Reading r = items[head];
    head = (head + 1) % items.length;
    count--;
    notifyAll();                   // wake producers waiting for space
    return r;
}
```

What I did (say this)

> "I wrote the bounded buffer between the five sensor threads and the consumer thread as a Java monitor – synchronized methods, wait in a while loop and notifyAll – without using BlockingQueue. I wrote the consumer thread, which stops cleanly when it is interrupted. I checked every piece of shared data in the project and protected it with synchronized, volatile or atomic variables, documented in our shared-data table. Finally I built the race-condition demo: the unsafe counter lost about 70 000 of 200 000 updates, while both fixes are always exactly 200 000."

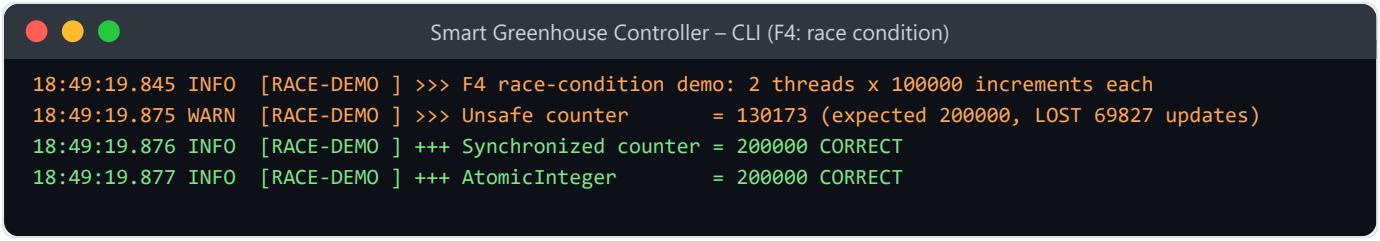
Demo

- **Postman:** run **F4-RaceCondition**, or press menu `4`.
- **CLI shows:**

```
>>> Unsafe counter       = 130173 (expected 200000, LOST 69827 updates)
+++ Synchronized counter = 200000 CORRECT
+++ AtomicInteger        = 200000 CORRECT
```

- **Point at the code:** `BoundedBuffer.put()` / `take()`. Explain `while` vs `if`.
- **Show:** the shared-data table.

What the CLI looks like:



Questions the lecturer may ask

Question	Answer
Why `while` and not `if` around `wait()`?	A thread can wake up without the condition being true (spurious wake-up, or another thread took the item first). It must check again.
Why `notifyAll()` and not `notify()`?	Producers and consumers wait on the same monitor. `notify()` could wake the wrong kind of thread, so nobody makes progress.
Difference between `volatile` and `synchronized`?	`volatile` gives visibility of one variable only. `synchronized` gives visibility plus mutual exclusion for a block of code.
Why is `AtomicInteger` correct without a lock?	`incrementAndGet()` is one atomic compare-and-set operation in the CPU.
Why does the unsafe result change every run?	The race depends on the exact timing of the two threads.

Report section

Shared-data table (what is shared, who uses it, how it is protected) in CONTRIBUTIONS.md, Role 3.

Role 4 – Control Engineer

Job

Decide what the greenhouse does. Role 4 owns the control rules for heater, fan, pump and alarm, the fast alarm reaction, and the actuators that log every change.

Code

File	Purpose
`control/Controller.java`	Control cycle every 1 s, rules, Observe & React, RT2 measurement
`control/Actuator.java`	One actuator. Logs every change with a timestamp and a reason.
`control/Actuators.java`	Heater, fan, pump and alarm together
Postman	`01-Normal-Operation`, `F5-HeatWave`, `X3-HumiditySoilFailure`

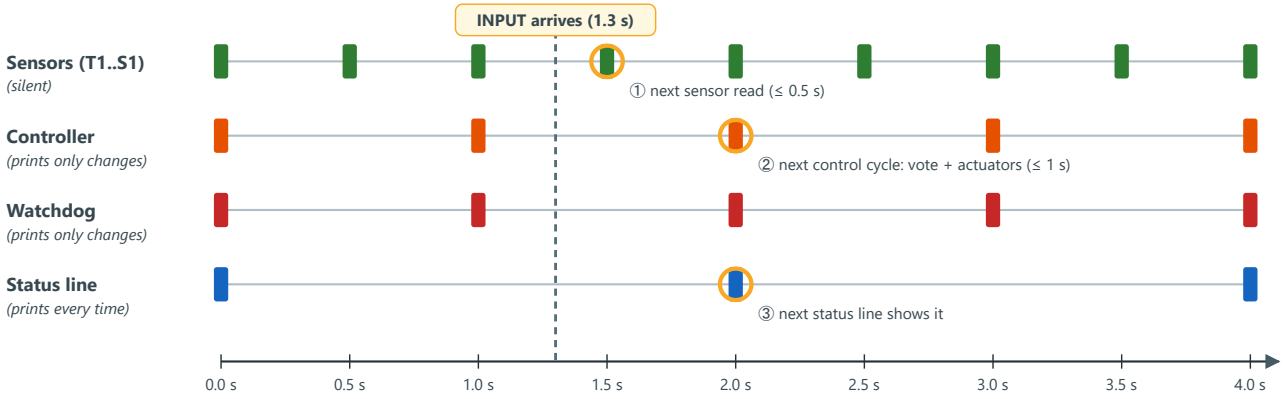
How it works (process)

1. **Environmental Control (periodic, every 1 s, RT4 firm).** Each cycle runs:
    1. collect the trustworthy readings;
    2. vote (Role 2);
    3. if the vote is FAILED → safe state (Role 2);
    4. otherwise apply the rules.
2. **Rules:**

Actuator	Rule
Heater	ON below 22 °C, OFF above 24 °C. Between 22 and 24 it **keeps its state** (hysteresis, so it doesn't switch on/off every second).
Fan	ON above 28 °C or humidity above 85 %
Pump	ON below 35 % soil moisture, OFF above 50 % (hysteresis)
Alarm	ON above 35 °C

3. **Sensors without redundancy:**
    - Humidity sensor failed → fan ON as a precaution.
    - Soil sensor failed → pump OFF, so it can't flood.
    - Both repeat a warning every cycle.
4. **Observe & React (event-driven, RT2 hard 1 s).**
    - Waiting for the 1 s cycle could take 500 ms (sensor) + 1000 ms (cycle) = 1.5 s, which is too slow for a hard 1 s deadline.
    - So the consumer calls `onTemperatureReading()` for **every** new temperature. If the voted value is above 35 °C, the alarm and fan switch ON immediately.
    - The reaction time is measured and logged (`[RT2 HARD] alarm ON 37 ms ... MET`).
5. **Robust cycle.** The whole cycle is inside `try/catch`. An exception → safe state, and the task keeps running next second (concept 5, together with Role 2).
6. **Actuator.** `set(state, reason)` only acts on a real change and logs it with a ms timestamp, e.g. `ALARM -> ON (T=39.0 C > 35.0 C)`.

The system is a loop that never stops – and an input in the middle



Reaction time: at most 0.5 s (sensor) + 1 s (control cycle). The alarm uses Observe & React, so it does not wait for the control cycle.

```
// Controller.controlClimate()
if (t < HEATER_ON_BELOW)       actuators.heater.set(true,  "T < 22 C");
else if (t > HEATER_OFF_ABOVE) actuators.heater.set(false, "T > 24 C");   // 22..24: keep state
actuators.fan.set(t > FAN_ON_ABOVE_TEMP || humidity > FAN_ON_ABOVE_HUMIDITY, ...);
actuators.alarm.set(t > ALARM_ABOVE, ...);
```

What I did (say this)

> "I built the controller using two patterns. Environmental Control is a periodic loop every second that reads, decides and switches heater, fan and pump with on/off limits and hysteresis. Observe & React handles the alarm: every new temperature reading is checked immediately, because waiting for the next cycle could take 1.5 seconds and miss the hard 1-second deadline. In our test the alarm switched on 37 ms after the heat wave. Every actuator change is logged with a millisecond timestamp and its reason."

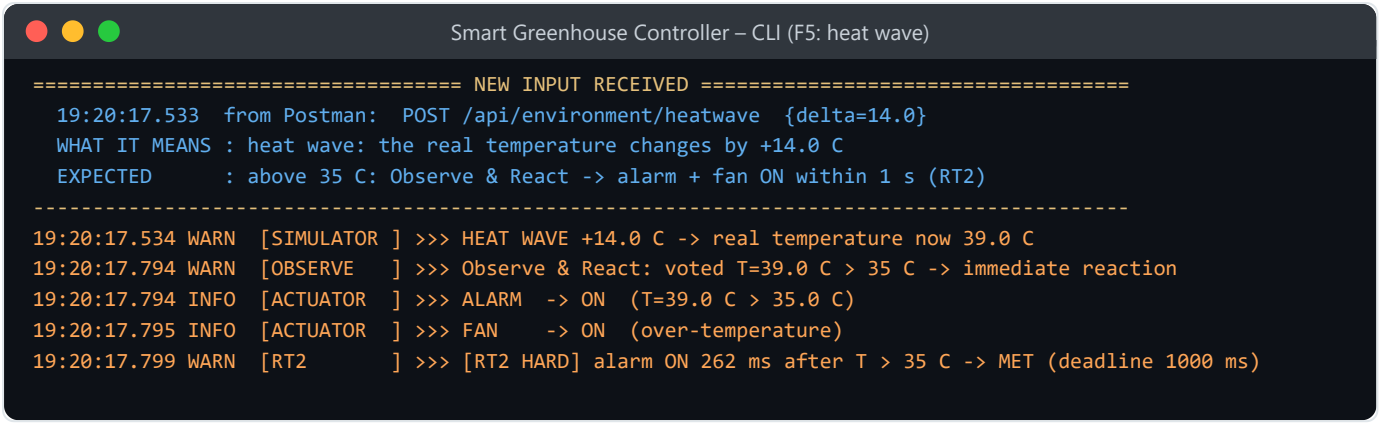
Demo

- **F5:** Postman **F5-HeatWave**, or menu `5`. CLI:

  ```
  >>> HEAT WAVE +14.0 C -> real temperature now 39.0 C
  >>> Observe & React: voted T=39.0 C > 35 C -> immediate reaction
  >>> ALARM  -> ON  (T=39.0 C > 35.0 C)
  >>> FAN    -> ON  (over-temperature)
  >>> [RT2 HARD] alarm ON 37 ms after temperature went above 35 C -> MET (deadline 1000 ms)
  ```

- **01-Normal-Operation:** heater, fan and pump switching, and the hysteresis.
- **X3:** soil sensor dead → pump OFF; humidity sensor dead → fan ON.
- **Point at the code:** `Controller.controlClimate()` and `Controller.onTemperatureReading()`.

What the CLI looks like:



Questions the lecturer may ask

Question	Answer
Difference between Environmental Control and Observe & React?	Environmental Control is periodic: check everything every 1 s. Observe & React is event-driven: react as soon as a specific stimulus is observed.
Why hysteresis?	Without it the heater would switch on/off every second around 22 °C, which wears out the hardware.
How do you know RT2 is met?	The time from the temperature crossing 35 °C to the alarm switching on is measured and logged: 37 ms, against a 1000 ms deadline.
Why pump OFF when the soil sensor fails?	Too much water damages plants faster than one dry period does, so OFF is the safe side.

Report section

Environmental Control and Observe & React patterns, and the stimulus–response table (in CONTRIBUTIONS.md, Role 4).

Role 5 – Timing & Integration Lead

Job

Put everything together and run it. Role 5 owns the main class and the kernel that starts and stops all threads, the status display, the console menu, logging, timing measurement and the RM test. It also owns the REST API for Postman, the scripts, the README and the group coordination.

Code

File	Purpose
`GreenhouseApplication.java`	Main class
`kernel/RealTimeKernel.java`	Builds everything, starts all periodic tasks, clean shutdown
`monitor/TimingMonitor.java`	Measures C, computes U, RM test, deadline misses
`monitor/StatusDisplay.java`	Status line every 2 s
`monitor/FaultMenu.java`	Console menu
`monitor/Log.java`, `monitor/EventRecorder.java`, `monitor/InputDescriber.java`	ms timestamps, log files, `NEW INPUT RECEIVED` boxes, banners, scenario summaries, start-up legend
`api/GreenhouseRestController.java`, `api/dto/ScenarioRequest.java`	REST endpoints for Postman
Postman	`00-Common-Setup`, `02-Repair-All`, `03-Timing-And-Shutdown`, `X4-MixedChaos`, environments, `build-collection.js`
Build / run	`pom.xml`, `mvnw*`, `run.bat`, `run.sh`, `run-demo.bat`, `test-*.bat`, `README.md`

How it works (process)

1. **Start-up.** `run.bat` finds Java 21, builds with the Maven wrapper and starts the jar. When the web server is ready, `RealTimeKernel.start()`:
   - creates one `ScheduledExecutorService` with 8 threads;
   - releases each task with `scheduleAtFixedRate`:

Task	Period
5 sensors	500 ms
Controller	1000 ms
Watchdog	1000 ms
Status display	2000 ms

   - starts the consumer thread and the menu thread.
2. **Timing measurement (concept 11).** Every task measures its own execution time:

```
long start = System.nanoTime();
... task body ...
timing.record(Task.CONTROL, "controller", start, System.nanoTime());
```

`TimingMonitor` keeps the **maximum C** (the estimate of the WCET), computes **U = C / T** per task and the total U, and compares it with the **RM bound n(2^(1/n) − 1) = 0.757** for n = 4. Late releases or overruns are logged as `DEADLINE MISS`.

3. **Clean shutdown** (menu `0` or Postman `03`):
    1. `executor.shutdown()` – no new releases;
    2. `awaitTermination()`;
    3. actuators to the safe position;
    4. `consumer.interrupt()` + `join()`;
    5. final timing report;
    6. close the log files.
4. **Logging.** Every important event has a ms timestamp and goes to:
    - the CLI;
    - `logs/<date>/greenhouse_<time>.log` (everything);
    - `logs/<date>/events_<time>.log` (important events only).

    Each Postman folder starts with a **banner** and ends with a numbered **summary**.
5. **Integration and testing.**
    - The REST API lets Postman play the "outside world".
    - The full collection (13 folders, 144 requests, 251 checks) runs automatically with `run-demo.bat`.
    - `X4-MixedChaos` tests all roles' features together.

What I did (say this)

> "I integrated the system. The main class starts all periodic tasks with a ScheduledExecutorService at a fixed rate and shuts them down cleanly with shutdown, awaitTermination and interrupt. Every task measures its execution time with System.nanoTime. I collect the maximum, compute the utilisation and run the Rate-Monotonic test. Our total U is about 0.014, far below the bound of 0.757, so the task set is schedulable – but normal Java cannot guarantee it, because of garbage collection and OS scheduling. I also built the status display, the console menu, the logging, the REST API, the Postman scripts and the one-command run scripts."

Demo

- **Start:** `run.bat`. Show the status line every 2 s.
- **Full automatic run:** `run-demo.bat slow`, or Postman → Run collection with the **Greenhouse-Presentation** environment.
- **End:** Postman **03-Timing-And-Shutdown**, or menu `0`. CLI:

```
================ TIMING REPORT (FINAL, 154 s) ================
Task              T(ms)    runs   avgC(ms)   maxC(ms)      U=C/T
Sensor sampling     500    1545     0.0191     2.2372   0.004474
Control cycle      1000     154     0.3357     3.6274   0.003627
Watchdog check     1000     154     0.0618     3.1581   0.003158
Status display     2000      77     0.4935     6.0596   0.003030
Total U = 0.014290   RM bound n=4 = 0.757   -> SCHEDULABLE
Deadline misses so far: 0
Clean shutdown complete - all real-time threads stopped
```

- **Point at the code:** `RealTimeKernel.start()` / `stop()` and `TimingMonitor.record()`.

What the CLI looks like:

Smart Greenhouse Controller – CLI (final timing report)

```
18:50:03.898 INFO  [TIMING    ] ================ TIMING REPORT (FINAL, 154 s) ================
18:50:03.898 INFO  [TIMING    ] Task              T(ms)    runs   avgC(ms)   maxC(ms)      U=C/T
18:50:03.898 INFO  [TIMING    ] Sensor sampling     500    1545     0.0191     2.2372   0.004474
18:50:03.899 INFO  [TIMING    ] Control cycle      1000     154     0.3357     3.6274   0.003627
18:50:03.899 INFO  [TIMING    ] Watchdog check     1000     154     0.0618     3.1581   0.003158
18:50:03.899 INFO  [TIMING    ] Status display     2000      77     0.4935     6.0596   0.003030
18:50:03.899 INFO  [TIMING    ] Total U = 0.014290   RM bound n=4 = 0.757   -> SCHEDULABLE
18:50:03.899 INFO  [TIMING    ] Deadline misses so far: 0
18:50:03.899 INFO  [KERNEL    ] +++ Clean shutdown complete - all real-time threads stopped
```

Questions the lecturer may ask

Question	Answer
What does U = 0.014 mean?	The 4 tasks use about 1.4 % of the CPU in the worst measured case.
Why compare with 0.757?	Liu & Layland: with Rate-Monotonic priorities, n tasks always meet their deadlines if U ≤ n(2^(1/n) − 1). For n = 4 that is 0.757.
Is the measured max C the real WCET?	No, it is only an estimate from measurements. A real WCET needs static analysis.
Why can't normal Java guarantee the deadlines?	Garbage-collection pauses, the OS scheduler (Java priorities are only hints) and JIT warm-up. A real-time OS or Real-Time Java is needed for guarantees.
How do you stop threads cleanly?	`shutdown()` stops new releases, `awaitTermination()` waits, `interrupt()` wakes the consumer out of `wait()`, and `join()` waits for it to end.

Report section

Section 4 timing table, RM test, why Java can't guarantee deadlines, README and run instructions.

Presentation running order (about 15 minutes)

#	Who	Time	What
1	Role 5	2 min	Problem, architecture picture (Section 0), start `run.bat`, show the status line
2	Role 1	2 min	Sensors and simulation → demo **F1**
3	Role 2	3 min	Voter, watchdog, safe state → demo **F2** and **F3**
4	Role 3	2.5 min	Monitor buffer, shared-data table → demo **F4**
5	Role 4	2.5 min	Control patterns → demo **F5**
6	Role 2 / 5	1 min	Robustness → **X1** (invalid inputs) or **X4** (mixed chaos)
7	Role 5	2 min	Shutdown → timing table, RM test, Java limits, conclusion

Tips for the demo:

- Use the **Greenhouse-Presentation** environment (+3 s per step), so the audience can follow.
- Put Postman and the CLI side by side, maximize the CLI, and don't click inside it (that pauses the output).
- After each folder, read the numbered **SUMMARY** out loud. It tells the whole story of the scenario.
- If something goes wrong live: menu `r` resets the system, and every folder starts with a reset anyway.